

Projeto 2 - System Calls

Participantes:

[Pedro Arthur Santana Patriota](#)

[Yasmim Vitoria Silva de Oliveira](#)

[Pedro Augusto de Almada Lobo Fonseca](#)

[Arthur Brito Medeiros](#)

1. Decisões de implementação

1.1 Dispatcher de syscalls e validação de memória

A implementação principal ficou em **pintos/src/userprog/syscall.c**. O **syscall_handler()** lê o número da syscall em **f->esp** e busca os argumentos na pilha do usuário seguindo a convenção x86: **f->esp** contém o número da syscall, e **f->esp + 4**, **f->esp + 8** e **f->esp + 12** contêm os argumentos.

Antes de qualquer leitura, os endereços são validados com **is_user_vaddr()** e **pagedir_get_page()**. A leitura de inteiros de 32 bits é feita byte a byte, o que protege casos em que o argumento cruza uma fronteira de página. Strings vindas do usuário, como nomes de arquivos e linhas de comando em **exec()**, são copiadas para memória do kernel para evitar uso direto de ponteiros de user space durante a syscall.

Buffers de **read()** e **write()** também são tratados byte a byte. Em **write()**, o kernel copia dados do usuário para um buffer temporário antes de chamar **putbuf()** ou **file_write()**. Em **read()**, os dados são lidos para um buffer do kernel e depois copiados para o buffer do usuário.

Para buffers em que o kernel escreve de volta no espaço de usuário, como o buffer de **read()**, a validação também verifica se a página está marcada como gravável na page table. Foi adicionada a consulta **pagedir_is_writable()**, que checa o bit **PTE_W** da entrada correspondente. Isso evita que **copy_out()** escreva por um alias de kernel em páginas de usuário mapeadas como somente leitura, como segmentos de código.

Trecho central dessa validação:

```
static void
validate_user_writable_address (const uint8_t *uaddr)
{
    validate_user_address (uaddr);
    if (!pagedir_is_writable (thread_current ()->pagedir, uaddr))
        process_exit_with_status (-1);
}
```

Leitura segura de palavras de 32 bits, para cobrir argumentos que cruzam fronteiras de página:

```
static uint32_t
get_user_word (const void *uaddr)
{
    const uint8_t *bytes = uaddr;
    uint32_t word = 0;
    size_t i;

    for (i = 0; i < sizeof word; i++)
        word |= (uint32_t) get_user_byte (bytes + i) << (i * 8);
    return word;
}
```

Se um ponteiro inválido é detectado, o processo atual é encerrado com **exit(-1)**, sem derrubar o kernel.

1.2 Chamadas de sistema implementadas

Foram implementadas todas as syscalls do Projeto 2: **halt**, **exit**, **exec**, **wait**, **create**, **remove**, **open**, **filesize**, **read**, **write**, **seek**, **tell** e **close**.

```
case SYS_EXEC:
{
    const char *ucmd = (const char *)
get_user_word (esp + 1);
    char *cmd = copy_in_string
(ucmd);
    tid_t tid = TID_ERROR;

    if (cmd != NULL)
    {
        tid = process_execute (cmd);
        palloc_free_page (cmd);
    }
    f->eax = tid;
    break;
}
```

```
case SYS_CREATE:
{
    const char *ufile = (const char
*) get_user_word (esp + 1);
    unsigned initial_size =
(unsigned) get_user_word (esp + 2);
    char *file = copy_in_string
```

```
case SYS_WAIT:
{
    f->eax = process_wait ((tid_t)
get_user_word (esp + 1));
    break;
}
```

```
case SYS_REMOVE:
{
    const char *ufile = (const char
*) get_user_word (esp + 1);
    char *file = copy_in_string
(ufile);
    bool ok = false;
```

```

(ufile);
    bool ok = false;

    if (file != NULL)
    {
        syscall_filesys_lock_acquire
        ();
        ok = filesys_create (file,
initial_size);
        syscall_filesys_lock_release
        ();
        palloc_free_page (file);
    }
    f->eax = ok;
    break;
}

```

```

    if (file != NULL)
    {
        syscall_filesys_lock_acquire
        ();
        ok = filesys_remove (file);
        syscall_filesys_lock_release
        ();
        palloc_free_page (file);
    }
    f->eax = ok;
    break;
}

```

```

case SYS_OPEN:
{
    const char *ufile = (const char
*) get_user_word (esp + 1);
    char *file_name = copy_in_string
(ufile);
    int fd = -1;

    if (file_name != NULL)
    {
        struct file *file;

        syscall_filesys_lock_acquire
        ();
        file = filesys_open
(file_name);
        if (file != NULL)
        {
            fd = allocate_fd(file);
            if (fd == -1)
                file_close (file);
        }
        syscall_filesys_lock_release
        ();
        palloc_free_page (file_name);
    }
    f->eax = fd;
    break;
}

```

```

case SYS_FILESIZE:
{
    int fd = (int) get_user_word (esp
+ 1);
    struct file *file =
get_file_from_fd (fd);
    int size = -1;

    if (file != NULL)
    {
        syscall_filesys_lock_acquire
        ();
        size = file_length (file);
        syscall_filesys_lock_release
        ();
    }
    f->eax = size;
    break;
}

```

```

case SYS_READ:
{

```

```

case SYS_WRITE:
{

```

```

    int fd = (int) get_user_word (esp
+ 1);
    void *buffer = (void *)
get_user_word (esp + 2);
    unsigned size = (unsigned)
get_user_word (esp + 3);
    unsigned done = 0;

    if (size == 0)
    {
        f->eax = 0;
        break;
    }
    validate_user_writable_buffer
(buffer, size);

    if (fd == 0)
    {
        while (done < size)
        {
            uint8_t c = input_getc
();
            copy_out ((uint8_t *)
buffer + done, &c, 1);
            done++;
        }
        f->eax = done;
    }
    else
    {
        struct file *file =
get_file_from_fd (fd);

        if (file == NULL)
        {
            f->eax = -1;
            break;
        }

        while (done < size)
        {
            uint8_t chunk[128];
            unsigned chunk_size =
size - done;
            int bytes_read;

            if (chunk_size > sizeof
chunk)

```

```

    int fd = (int) get_user_word (esp
+ 1);
    const void *buffer = (const void
*) get_user_word (esp + 2);
    unsigned size = (unsigned)
get_user_word (esp + 3);
    unsigned done = 0;

    if (size == 0)
    {
        f->eax = 0;
        break;
    }
    validate_user_buffer (buffer,
size);

    if (fd == 1)
    {
        while (done < size)
        {
            uint8_t chunk[256];
            unsigned chunk_size =
size - done;

            if (chunk_size > sizeof
chunk)
                chunk_size = sizeof
chunk;

            copy_in (chunk, (const
uint8_t *) buffer + done,
chunk_size);
            putbuf ((const char *)
chunk, chunk_size);
            done += chunk_size;
        }
        f->eax = done;
    }
    else
    {
        struct file *file =
get_file_from_fd (fd);

        if (file == NULL)
        {
            f->eax = -1;
            break;
        }

```

```

        chunk_size = sizeof
chunk;

syscall_filesys_lock_acquire ();
        bytes_read = file_read
(file, chunk, chunk_size);

syscall_filesys_lock_release ();

        if (bytes_read <= 0)
            break;

        copy_out ((uint8_t *)
buffer + done, chunk, bytes_read);
        done += bytes_read;
        if ((unsigned) bytes_read
< chunk_size)
            break;
    }
    f->eax = done;
}
break;
}

```

```

case SYS_SEEK:
{
    int fd = (int) get_user_word (esp
+ 1);
    unsigned position = (unsigned)
get_user_word (esp + 2);
    struct file *file =
get_file_from_fd (fd);

    if (file != NULL)
    {
        syscall_filesys_lock_acquire
();
        file_seek (file, position);
    }
}

```

```

        while (done < size)
        {
            uint8_t chunk[128];
            unsigned chunk_size =
size - done;
            int bytes_written;

            if (chunk_size > sizeof
chunk)

                chunk_size = sizeof
chunk;
            copy_in (chunk, (const
uint8_t *) buffer + done,
chunk_size);

            syscall_filesys_lock_acquire ();
            bytes_written =
file_write (file, chunk, chunk_size);

            syscall_filesys_lock_release ();

            if (bytes_written <= 0)
                break;
            done += bytes_written;
            if ((unsigned)
bytes_written < chunk_size)
                break;
        }
        f->eax = done;
    }
    break;
}

```

```

case SYS_TELL:
{
    int fd = (int) get_user_word (esp
+ 1);
    struct file *file =
get_file_from_fd (fd);
    unsigned position = 0;

    if (file != NULL)
    {
        syscall_filesys_lock_acquire
();
        position = file_tell (file);
        syscall_filesys_lock_release

```

<pre> syscall_filesys_lock_release); } break; } </pre>	<pre>); } f->eax = position; break; } </pre>
---	--

Os descritores **0** e **1** foram reservados para **stdin** e **stdout**. A syscall **read()** usa **input_getc()** quando recebe **fd = 0**, enquanto **write()** usa **putbuf()** quando recebe **fd = 1**. Arquivos comuns são acessados por uma tabela de descritores mantida em cada processo.

Cada processo recebeu uma tabela fixa de 128 descritores. As entradas **0** e **1** não são retornadas por **open()**, então arquivos abertos começam em **fd = 2**. Os descritores não são herdados por processos filhos, como esperado pelos testes do PintOS.

Para sustentar essa lógica, a **struct thread** recebeu campos de processo: lista de filhos, registro compartilhado com o pai, status de saída, tabela de file descriptors, próximo fd livre e ponteiro para o executável em execução.

Estruturas principais adicionadas em **thread.h**:

```

#define MAX_FD 128

struct child_process
{
    tid_t tid;
    int exit_status;
    bool load_success;
    bool waited;
    int ref_cnt;
    struct semaphore load_sema;
    struct semaphore exit_sema;
    struct list_elem elem;
};

struct file *fd_table[MAX_FD];
struct file *executable;

```

Trecho simplificado da alocação de descritores:

```

for (i = 2; i < MAX_FD; i++)
{
    int fd = cur->next_fd;
    if (cur->fd_table[fd] == NULL)
    {
        cur->fd_table[fd] = file;
    }
}

```

```

        cur->next_fd = fd + 1;
        return fd;
    }
}

```

1.3 Sincronização do sistema de arquivos

O sistema de arquivos base do PintOS não é thread-safe. Por isso, foi criado um lock global em **syscall.c**, inicializado em **syscall_init()**, para proteger as chamadas a **filesys_*** e **file_***.

Esse lock é usado em operações de criação, remoção, abertura, leitura, escrita, consulta de tamanho, busca de posição, fechamento de arquivos, carregamento de executáveis e limpeza em **process_exit()**. A solução reduz paralelismo, mas é adequada ao Projeto 2 porque evita corridas e inconsistências nos testes **syn-***.

Wrappers usados também por **process.c** durante o carregamento e fechamento de executáveis:

```

void
syscall_filesys_lock_acquire (void)
{
    lock_acquire (&filesys_lock);
}

```

```

void
syscall_filesys_lock_release (void)
{
    lock_release (&filesys_lock);
}

```

1.4 Processos, **exec()**, **wait()** e limpeza

A implementação anterior de **process_wait()** era provisória e usava espera ocupada. Ela foi substituída por sincronização real entre pai e filho. Para isso, foi adicionada a estrutura **child_process**, armazenada na lista de filhos do pai.

Essa estrutura guarda o **tid** do filho, status de saída, resultado do carregamento, flag de espera única, semáforo para sincronizar o fim de **load()**, semáforo para sincronizar o término do processo e contador de referências. O contador permite liberar o registro corretamente mesmo quando pai e filho terminam em ordens diferentes.

Com isso, **exec()** só retorna ao processo pai depois que o filho informa se conseguiu carregar o executável. Se o carregamento falha, **exec()** retorna **-1**. Se carrega com sucesso, retorna o **tid** usado como **pid**.

O ponto principal da sincronização é o pai bloquear até o filho sinalizar o resultado de **load()**:

```

tid = thread_create (prog_name, PRI_DEFAULT, start_process, args);

```

```
child->tid = tid;
sema_down (&child->load_sema);
if (!child->load_success)
    return TID_ERROR;
```

wait() agora aceita apenas filhos diretos, permite esperar cada filho uma única vez e retorna o status informado por **exit()**. Se o filho morre por uma exceção de usuário, o status retornado é **-1**.

Trecho resumido do controle de espera única:

```
if (child->waited)
    return -1;

child->waited = true;
sema_down (&child->exit_sema);
status = child->exit_status;
```

Foi adicionada uma função **process_exit_with_status()**, usada por **exit()** e por exceções de user mode. Ela registra o status de saída, imprime a mensagem no formato esperado:

```
printf ("%s: exit(%d)\n", ...);
```

Depois disso, chama **thread_exit()**, que executa **process_exit()**.

Em **process_exit()**, o kernel fecha todos os arquivos abertos, fecha o executável mantido aberto, libera registros de filhos órfãos, sinaliza o pai quando necessário e destrói a page table do processo. Essa limpeza é importante para **multi-oom**, porque o teste cria muitos processos e alguns morrem de forma anormal depois de abrir vários arquivos.

Parte da limpeza de descritores e executável:

```
for (fd = 2; fd < MAX_FD; fd++)
    if (cur->fd_table[fd] != NULL)
        file_close (cur->fd_table[fd]);

if (cur->executable != NULL)
    file_close (cur->executable);
```

1.5 Proteção contra escrita no executável

Durante **load()**, o executável do processo é mantido aberto em **thread_current()->executable** e recebe **file_deny_write()**. O arquivo só é fechado em **process_exit()**.

Essa decisão impede que um processo sobrescreva o próprio binário enquanto ele está executando e cobre os testes **rox-simple**, **rox-child** e **rox-multichild**.

Trecho aplicado após o carregamento bem-sucedido:

```
file_deny_write (file);  
t->executable = file;  
file = NULL;
```

1.6 Tratamento de exceções de usuário

pintos/src/userprog/exception.c foi ajustado para que exceções originadas em user mode encerrem apenas o processo causador com **exit(-1)**. Antes, o handler imprimia dumps de interrupção e encerrava a thread sem registrar corretamente o status esperado.

Com a alteração, acessos inválidos como leitura de **NULL**, escrita em endereço inválido ou salto para endereço não mapeado não causam panic no kernel e retornam o comportamento esperado pelos testes.

Trecho aplicado para exceções de user mode:

```
case SEL_UCSEG:  
    process_exit_with_status (-1);  
  
if (user)  
    process_exit_with_status (-1);
```

2. Testes

Todos os testes ligados à passagem de argumentos (**args-***), system calls (**sc-***, **halt**, **exit**), arquivos (**create-***, **open-***, **close-***, **read-***, **write-***), processos (**exec-***, **wait-***, **multi-recurse**, **multi-child-fd**), proteção do executável (**rox-***), acessos inválidos (**bad-***), pressão de memória e limpeza de recursos (**no-vm/multi-oom**) e filesystem base passaram. Também foram repetidos os testes de sincronização do filesystem (**syn-read**, **syn-remove** e **syn-write**), que passaram em execuções adicionais.

```

pass tests/userprog/args-none
pass tests/userprog/args-single
pass tests/userprog/args-multiple
pass tests/userprog/args-many
pass tests/userprog/args-dbl-space
pass tests/userprog/sc-bad-sp
pass tests/userprog/sc-bad-arg
pass tests/userprog/sc-boundary
pass tests/userprog/sc-boundary-2
pass tests/userprog/sc-boundary-3
pass tests/userprog/halt
pass tests/userprog/exit
pass tests/userprog/create-normal
pass tests/userprog/create-empty
pass tests/userprog/create-null
pass tests/userprog/create-bad-ptr
pass tests/userprog/create-long
pass tests/userprog/create-exists
pass tests/userprog/create-bound
pass tests/userprog/open-normal
pass tests/userprog/open-missing
pass tests/userprog/open-boundary
pass tests/userprog/open-empty
pass tests/userprog/open-null
pass tests/userprog/open-bad-ptr
pass tests/userprog/open-twice
pass tests/userprog/close-normal
pass tests/userprog/close-twice
pass tests/userprog/close-stdin
pass tests/userprog/close-stdout
pass tests/userprog/close-bad-fd
pass tests/userprog/read-normal
pass tests/userprog/read-bad-ptr
pass tests/userprog/read-boundary
pass tests/userprog/read-zero
pass tests/userprog/read-stdout
pass tests/userprog/read-bad-fd
pass tests/userprog/write-normal
pass tests/userprog/write-bad-ptr
pass tests/userprog/write-boundary

```

```

pass tests/userprog/write-zero
pass tests/userprog/write-stdin
pass tests/userprog/write-bad-fd
pass tests/userprog/exec-once
pass tests/userprog/exec-arg
pass tests/userprog/exec-bound
pass tests/userprog/exec-bound-2
pass tests/userprog/exec-bound-3
pass tests/userprog/exec-multiple
pass tests/userprog/exec-missing
pass tests/userprog/exec-bad-ptr
pass tests/userprog/wait-simple
pass tests/userprog/wait-twice
pass tests/userprog/wait-killed
pass tests/userprog/wait-bad-pid
pass tests/userprog/multi-recurse
pass tests/userprog/multi-child-fd
pass tests/userprog/rox-simple
pass tests/userprog/rox-child
pass tests/userprog/rox-multichild
pass tests/userprog/bad-read
pass tests/userprog/bad-write
pass tests/userprog/bad-read2
pass tests/userprog/bad-write2
pass tests/userprog/bad-jump
pass tests/userprog/bad-jump2
pass tests/userprog/no-vm/multi-oom
pass tests/filesys/base/lg-create
pass tests/filesys/base/lg-full
pass tests/filesys/base/lg-random
pass tests/filesys/base/lg-seq-block
pass tests/filesys/base/lg-seq-random
pass tests/filesys/base/sm-create
pass tests/filesys/base/sm-full
pass tests/filesys/base/sm-random
pass tests/filesys/base/sm-seq-block
pass tests/filesys/base/sm-seq-random
pass tests/filesys/base/syn-read
pass tests/filesys/base/syn-remove
pass tests/filesys/base/syn-write
All 80 tests passed.

```

3. Arquivos alterados e adicionados

Arquivos alterados:

- pintos/src/threads/thread.c e pintos/src/threads/thread.h
- pintos/src/userprog/exception.c
- pintos/src/userprog/process.c e pintos/src/userprog/process.h
- pintos/src/userprog/pagedir.c e pintos/src/userprog/pagedir.h
- pintos/src/userprog/syscall.c e pintos/src/userprog/syscall.h